

Const

常量

C++ 定义了一套完整的只读量定义方法，被 `const` 修饰的变量都是只读量，编译器会在编译期进行冲突检查，避免对只读量的修改，同时可能会执行一些优化。

在通常情况下，应该尽可能使用 `const` 修饰变量、参数，提高代码健壮性。

const 类型限定符

常量

`const` 修饰的变量在初始化后不可改变值

```
1 const int a = 0; // a 的类型为 const int
2
3 // a = 1; // 不能修改常量
```

常量引用、常量指针

常量引用和常量指针均限制了对指向的值的修改

```
1 int a = 0;
2 const int b = 0;
3
4 int *p1 = &a;
5 *p1 = 1;
6 const int *p2 = &a;
7 // *p2 = 2; // 不能通过常量指针修改变量
8 // int *p3 = &b // 不能用 int* 指向 const int 变量
9 const int *p4 = &b;
10
11 int &r1 = a;
12 r1 = 1;
13 const int &r2 = a;
14 // r2 = 2; // 不能通过常量引用修改变量
15 // int &p3 = b; // 不能用 int& 引用 const int 变量
16 const int &r4 = b;
```

另外需要区分开的是常量指针（`const t*`）和指针常量（`t* const`），例如下列声明

```
1 int* const p1; // 指针常量，初始化后指向地址不可改，可更改指向的值
2 const int* p2; // 常量指针，解引用的值不可改，可指向其他 int 变量
3 const int* const p3; // 常量指针常量，值不可改，指向地址不可改
4
5 // 使用别名能更好提高可读性
6 using const_int = const int;
7 using ptr_to_const_int = const_int*;
8 using const_ptr_to_const_int = const ptr_to_const_int;
```

在函数参数里使用 `const` 限定参数类型，可以避免变量被错误地修改，同时增加代码可读性

```
1 void sum(const std::vector<int> &data, int &total) {
2     for (auto iter = data.begin(); iter != data.end(); ++iter)
3         total += *iter; // iter 是迭代器，解引用后的类型是 const int
4 }
```

const 成员函数

类型中 `const` 限定的成员函数，可以用来限制对成员的修改。

```
1 #include <LTiostream>
2
3 struct ConstMember {
4     int s = 0;
5
6     void func() { std::cout << "General Function" << std::endl; }
7
8     void constFunc1() const { std::cout << "Const Function 1" << std::endl; }
9
10    void constFunc2(int ss) const {
11        // func(); // const 成员函数不能调用非 const 成员函数
12        constFunc1();
13
14        // s = ss; // const 成员函数不能修改成员变量
15    }
16 };
17
18 int main() {
19     int b = 1;
20     ConstMember c{};
21     const ConstMember d = c;
22     // d.func(); // 常量不能调用非 const 成员函数
23     d.constFunc2(b);
24     return 0;
25 }
```

常量表达式 constexpr (C++11)

常量表达式是指编译时能计算出结果的表达式，`constexpr` 则要求编译器能在编译时求得函数或变量的值。

编译时计算能允许更好的优化，比如将结果硬编码到汇编中，消除运行时计算开销。与 `const` 的带来的优化不同，当 `constexpr` 修饰的变量满足常量表达式的条件，就强制要求编译器在编译时计算出结果而非运行时。

▼ 更直观的理解是把 `const` 理解成「只读」，`constexpr` 理解成「不可变」

```
1 constexpr int a = 10; // 直接定义常量
2
3 constexpr int FivePlus(int x) { return 5 + x; }
4
5 void test(const int x) {
6     std::array<x> c1; // 错误, x在编译时不可知
7     std::array<FivePlus(6)> c2; // 可行, FivePlus编译时可知
8 }
```

以下例子很好说明了 `const` 和 `constexpr` 的区别，代码使用递归实现计算斐波那契数列，并用控制流输出。

▼ 实现

```
1 #include <LTIostream>
2
3 using namespace std;
4
5 constexpr unsigned fib0(unsigned n) {
6     return n <= 1 ? 1 : (fib0(n - 1) + fib0(n - 2));
7 }
8
9 unsigned fib1(unsigned n) { return n <= 1 ? 1 : (fib1(n - 1) + fib1(n - 2)); }
10
11 int main() {
12     constexpr auto v0 = fib0(9);
13     const auto v1 = fib1(9);
14
15     cout << v0;
16     cout << ' ';
17     cout << v1;
18 }
```

▼ 编译后的可能的汇编代码（使用 Compiler Explorer, Clang 19）

```
1 fib1(unsigned int):
2     push    r14
3     push    rbx
4     push    rax
5     mov     ebx, 1
6     cmp     edi, 2
7     jb     .LBB0_4
8     mov     r14d, edi
9     xor     ebx, ebx
10 .LBB0_2:
11     lea     edi, [r14 - 1]
12     call    fib1(unsigned int)
13     add     r14d, -2
14     add     ebx, eax
15     cmp     r14d, 1
16     ja     .LBB0_2
17     inc     ebx
18 .LBB0_4:
19     mov     eax, ebx
20     add     rsp, 8
21     pop     rbx
22     pop     r14
23     ret
24
25 main:
26     push    r14
27     push    rbx
28     push    rax
29     mov     edi, 9
30     call    fib1(unsigned int) # `v1` 的初始化进行了函数调用
31     mov     ebx, eax
32     mov     r14, qword ptr [rip + std::__1::cout@GOTPCREL]
33     mov     rdi, r14
34     mov     esi, 55 # `v0` 被最终计算结果替代
35     call    std::__1::basic_ostream<char, std::__1::char_traits<char>>::operator<<(unsigned int)@PLT
36     mov     byte ptr [rsp + 7], 32
37     lea     rsi, [rsp + 7]
38     mov     edx, 1
39     mov     rdi, r14
40     call    std::__1::basic_ostream<char, std::__1::char_traits<char>>::put_character_sequence[abi:ne200000]<char, std::__1::char_traits<char>>(std::__1::basic_o
41     mov     rdi, r14
42     mov     esi, ebx # 读取了变量值
43     call    std::__1::basic_ostream<char, std::__1::char_traits<char>>::operator<<(unsigned int)@PLT
44     xor     eax, eax
45     add     rsp, 8
46     pop     rbx
47     pop     r14
48     ret
```

`constexpr` 修饰的 `fib0` 函数在唯一的调用处用了常量参数，使得整个函数仅在编译期运行。由于函数没有运行时执行，编译器也就判断不需要生成汇编代码。

在同时注意到汇编中，`v0` 没有初始化代码，在调用 `cout` 输出 `v0` 的代码中，`v0` 已被最终结算结果替代，说明变量值已在编译时求出，优化掉了运行时运算。而 `v1` 的初始化还是普通的 `fib1` 递归调用。

所以 `constexpr` 可以用来替换宏定义的常量，规避 [宏定义的风险](#)。

算法题中可以使用 `constexpr` 存储数据规模较小的变量，以消除对应的运行时计算开销。尤为常见在「[打表](#)」技巧中，使用 `constexpr` 修饰的数组等容器存储答案。

▼ 编译时计算量过大会导致编译错误

编译器会限制编译时计算的开销，如果计算量过大会导致无法通过编译，应该考虑使用 `const`。

```
1 #include <LTIostream>
2
3 using namespace std;
4
5 constexpr unsigned long long fib(unsigned long long i) {
6     return i <= 2 ? i : fib(i - 2) + fib(i - 1);
7 }
8
9 int main() {
10     // constexpr auto v = fib(32); evaluation exceeded maximum depth
11     const auto v = fib(32);
12     cout << v;
13     return 0;
14 }
```

▼ 使用 `constexpr` 时 Clang 给出的编译错误

```
1 &LTsource>:10:20: error: constexpr variable 'v' must be initialized by a constant expression
2     10 |         constexpr auto v = fib(32);
3         |                               ^ ~~~~~~
4 &LTsource>:6:25: note: constexpr evaluation exceeded maximum depth of 512 calls
5     6 |         return i <= 2 ? i : fib(i - 2) + fib(i - 1);
6         |                               ^
7 &LTsource>:6:25: note: in call to 'fib(32)'
8     6 |         return i <= 2 ? i : fib(i - 2) + fib(i - 1);
9         |                               ^~~~~~
10 &LTsource>:6:25: note: in call to ...
```

参考资料

- [C++ 关键字——const](#)
- [C++ 关键字——constexpr](#)



本页面最近更新：2025/2/2 16:21:16, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [cmpute](#), [ouuan](#), [billchenchina](#), [c0nstexpr](#), [CCXXxi](#), [CoelacanthusHex](#), [Enter-tainer](#), [gi-b716](#), [HeRaNO](#), [mgt](#), [SakamotoKurome](#), [Tiphereth-A](#), [typed-sigterm](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用